Nome: Isabel Cristina Marras Salles RA: 188790

Visão geral

O programa simula o funcionamento do acesso à memória virtual em um sistema operacional com paginação sob demanda e uso de TLB (*Translation Lookaside Buffer*), implementado na classe MemorySimulator (mem_sim.py). A leitura dos endereços virtuais a partir do arquivo de entrada (*trace*) é feita em um loop principal no arquivo main.py, que invoca o método access_memory(virtual_address) para cada endereço requisitado pela CPU e, ao final, imprime as estatísticas acumuladas de acertos na TLB (tlb_hits), faltas na TLB (tlb_misses) e faltas de página (page_faults).

O fluxo de execução a cada acesso segue o modelo de tradução de endereços da MMU e do sistema operacional:

1. A partir do endereço virtual recebido, o simulador calcula o número da página (pg = virtual_address // page_size) e o deslocamento dentro da página (deslocamento = virtual_address % page_size).
2. Inicialmente, verifica-se se a página está presente na TLB (self.tlb). Caso seja encontrada (TLB Hit), incrementa-se o contador tlb_hits, atualiza-se a ordem de uso da página na TLB (que utiliza a política LRU conforme o enunciado) e também o estado de referência da página na memória física (de acordo com a política escolhida), retornando diretamente o endereço físico correspondente.
3. Caso a página não esteja na TLB (TLB Miss), incrementa-se tlb_misses e consulta-se a Tabela de Páginas(self.page_table):
    - Se a página já estiver presente na tabela de páginas, insere-se a tradução (pg, frame) na TLB e calcula-se o endereço físico.
    - Se a página não estiver na tabela de páginas, caracteriza-se um Page Fault (page_faults += 1). O simulador então busca um quadro livre em self.frames. Caso não haja quadros livres, aplica-se a política de substituição de páginas escolhida (LRU ou SecondChance) para liberar um frame, atualiza-se a tabela de páginas e, por fim, insere-se a nova página na TLB antes de retornar o endereço físico.

Implementação

Para manter o controle das páginas na memória física e implementar as políticas de substituição, além das estruturas base (self.tlb, self.page_table e self.frames), foram criadas duas estruturas auxiliares no construtor da classe:

- **self.lru_frames** (lista): armazena a fila de páginas atualmente presentes na memória física. No algoritmo LRU, essa lista mantém as páginas ordenadas da menos recentemente usada (índice 0) até a mais recentemente usada (final da lista). No algoritmo SecondChance, ela atua como a fila circular (FIFO) de páginas candidatas à substituição.
- **self.bit** (dicionário): utilizado na política SecondChance para mapear cada página na memória física ao seu respectivo bit de referência (0 ou 1).

Abaixo estão detalhadas as funções implementadas no simulador:

1. **access_memory(virtual_address)**: Método principal que coordena todo o fluxo de tradução do endereço virtual em físico. Ele percorre a TLB buscando a página. Em caso de *TLB Hit*, reposiciona a tupla (pg, frame) no final de self.tlb (usando pop(i) seguido de append) para manter a ordem do LRU da TLB, atualiza a referência da página na memória principal (lru_frames no caso do LRU, ou self.bit[pg] = 1 no SecondChance) e retorna o endereço físico. Em caso de *TLB Miss*, verifica se a página está na tabela de páginas (self.page_table). Se estiver, apenas atualiza sua referência na memória; se não estiver, contabiliza o *page fault* e chama alocacao_frame(pg). Ao final do tratamento do *miss*, chama insercao_tlb, para atualizar a tlb, e retorna o endereço físico.

2. **calcula_end_fisico(virtual_address, frame):** Função auxiliar responsável por montar o endereço físico a partir do endereço virtual e do número do frame onde a página está alocada. Calcula o deslocamento a partir do resto da divisão (virtual_address % self.page_size) e retorna *frame * self.page_size + deslocamento*.

3. **insercao_tlb(pg, frame):** Função auxiliar que insere o par (pg, frame) na TLB seguindo a política LRU. Como entradas recém-acessadas são sempre adicionadas ao final da lista self.tlb, o elemento mais antigo (menos recentemente usado) encontra-se sempre na posição 0. Assim, antes de inserir a nova tupla com append, a função verifica se a TLB atingiu sua capacidade máxima (len(self.tlb) == self.num_tlb_entries) e, se estiver cheia, remove o primeiro elemento com self.tlb.pop(0).

4. **alocacao_frame(pg):** Função chamada quando ocorre um *page fault* para alocar um frame na memória física para a página pg. Primeiro, ela percorre o vetor self.frames em busca de uma posição vazia (None). Se encontrar um frame livre i, armazena a páginas em em self.frames[i], registra o mapeamento na tabela de páginas (self.page_table[pg] = i), adiciona a página ao final de self.lru_frames e inicializa self.bit[pg] = 1. Caso todos os frames estejam ocupados, chama o algoritmo de método de substituição conforme o parâmetro self.rep_policy.

5. **LRU(pg)**: Implementa a substituição de páginas na memória física pelo algoritmo LRU (Least Recently Used*)*. Como a lista self.lru_frames é atualizada a cada acesso movendo a página referenciada para o final da lista, a página menos recentemente usada está sempre na posição 0. A função remove essa página (pg_remover = self.lru_frames.pop(0)), retira seu mapeamento antigo da tabela de páginas (self.page_table), obtendo o número do frame liberado (frame_remover), aloca a nova página pg nesse frame atualizando a referência na tabela de páginas (self.frames[frame_remover] = pg e self.page_table[pg] = frame_remover) e insere a páginas no final de self.lru_frames. Além disso, caso a página removida ainda conste na TLB, ela é removida para evitar acessos incorretos.

6. **SecondChance(pg)**: Implementa o algoritmo da Segunda Chance. A função percorre a fila self.lru_frames em um laço while True, inspecionando sempre a página na primeira posição da fila com a ordem de acessos passados (primeira_pg = self.lru_frames[0]):

- Se o bit de referência dessa página for 1 (self.bit[primeira_pg] == 1), ela recebe uma segunda chance. Seu bit é alterado para 0 e ela é movida do início para o final da fila (pop(0) seguido de append), passando a inspeção para a próxima página.
- Se o bit de referência for 0, a página é selecionada para ser removida (pg_remover = self.lru_frames.pop(0)). Ela é removida de self.page_table, de self.bit e de self.tlb (caso esteja presente), e a nova página pg assume o frame liberado com self.bit[pg] = 1, sendo adicionada ao final da fila.

Casos de testes adicionais

Foram criados dois novos conjuntos de teste na pasta tests/, ambos executados com page_size = 4096, num_tlb_entries = 16 e num_frames = 64:

1. **Acesso Aleatório (teste_aleatorio.in e saida_aleatorio.in)**

   Este caso de teste contém 1.000 acessos gerados de forma aleatória e distribuídos entre 256 páginas distintas. O objetivo é demonstrar como a ausência de localidade de referência prejudica o desempenho tanto da TLB quanto da memória virtual.
   Como resultado, de 1.000 acessos, ocorreram apenas 69 TLB Hits e 931 TLB Misses (taxa de acerto de apenas 6,9%, muito próxima da probabilidade teórica de 16/256 = 6,25% de uma página sorteada já estar na TLB). Da mesma forma, o número de *page faults* foi extremamente elevado em ambas as políticas: 745 Page Faults no LRU e 743 Page Faults no SecondChance (quase 75%, já que a memória física comporta apenas 64 das 256 páginas). Além disso, observa-se que LRU e SecondChance tiveram desempenho praticamente idêntico, pois quando os acessos são totalmente aleatórios, o histórico passado de uso não fornece informação útil para prever acessos futuros.

2. **Acesso com Localidade e Comparação LRU vs. SecondChance (teste_localidade.in e saida_localidade.in)**

   Este caso de teste possui 1.200 acessos que simulam o comportamento mais realista de um programa, combinando localidade espacial (acessos consecutivos a deslocamentos vizinhos dentro da mesma página antes de avançar para a próxima) e localidade temporal (um conjunto fixo de 32 páginas "principais" de 0 a 31 que são carregadas no início e frequentemente reutilizadas entre leituras de blocos de 32 páginas temporárias).
   Graças aos acessos seguidos dentro de cada página, o número de acertos na TLB subiu drasticamente para 800 TLB Hits, com 400 TLB Misses (66,7% de acerto na TLB, contra 6,9% do teste aleatório).
   Além disso, o LRU obteve 224 Page Faults, enquanto o SecondChance obteve 304 Page Faults (80 *page faults* a mais). Isso ocorre porque, quando os 64 quadros da memória enchem (com as páginas principais 0..31 e as temporárias 32..63) e o programa volta a acessar as páginas principais 0..31 antes da chegada de novas páginas:

- O LRU atualiza a ordem da fila movendo as páginas principais 0..31 para o final (mais recentes), fazendo com que as páginas temporárias 32..63 fiquem no início da fila e sejam descartadas primeiro, preservando as páginas principais na RAM.
- O SecondChance, por outro lado, apenas mantém bit = 1 para as páginas 0..31 sem alterar sua posição na fila (pois o bit de referência não guarda a ordem exata dos acessos). Como todas as 64 páginas ficam com bit = 1, quando um novo bloco de páginas chega, o algoritmo dá uma volta completa na fila zerando todos os bits e acaba substituindo justamente as páginas principais 0..31 que entraram primeiro (comportando-se como um FIFO nesse momento), o que gera novos *page faults* assim que o programa volta a requisitá-las.

Desafios encontrados

Os principais desafios encontrados durante a implementação foram em relação à adaptação do passo a passo visto em aula para a lógica do simulador e à manutenção conjunta das estruturas de dados da TLB e da memória física.

Um dos primeiros desafios foi definir o fluxo de retorno da função access_memory após ocorrer uma falta na TLB (*TLB Miss*). No algoritmo visto em aula, quando a página não é encontrada na TLB, o sistema operacional assume o controle, atualiza a TLB com a tradução da página e reinicia a instrução para que o processo volte a consultar a TLB. Inicialmente, tentar reproduzir esse retorno chamando a própria função access_memory(virtual_address) ao final do tratamento do *miss* fazia com que um único acesso contabilizasse tanto uma falta quanto um acerto na TLB na chamada seguinte. Para solucionar isso, o fluxo foi ajustado para que, após atualizar a tabela de páginas e inserir a página na TLB, o endereço físico já seja calculado e retornado diretamente, garantindo que cada endereço lido do arquivo de entrada corresponda a apenas um acesso contabilizado.

Outro ponto que demandou atenção foi modelar o algoritmo LRU para a memória física sem alterar a posição dos elementos dentro do vetor self.frames, visto que cada posição de self.frames representa um quadro físico fixo da RAM e não pode ser deslocada a cada acesso (como era feito na lista da TLB). A solução foi criar a lista auxiliar self.lru_frames para controlar apenas a ordem de uso das páginas que estão na memória e utilizar o dicionário self.page_table para localizar qual quadro físico de self.frames deveria ser atualizado no momento da substituição.

Por fim, na implementação do SecondChance, foi necessário cuidado para separar o comportamento de atualização em caso de acerto (*Hit*) em relação ao LRU: enquanto no LRU um acesso repetido exige remover a página do meio de self.lru_frames e inseri-la no final, no SecondChance a página deve permanecer na mesma posição da fila, atualizando apenas self.bit[pg] = 1. Também foi necessário garantir que, quando uma página vítima fosse substituída da memória física (especialmente no SecondChance, onde uma página recente pode ter seu bit zerado após um giro completo na fila), sua entrada também fosse

removida da TLB caso ainda estivesse lá, evitando que a TLB mantivesse referências para páginas que já haviam saído da RAM.